

메타태그 × 행동로그 기반 개인화 추천 쇼핑몰

2조 통모짜 개발자
이지수 전이레 정인혁

목차

1. 프로젝트 소개

1. 서비스 소개
2. 팀원 소개

2. 프로젝트 설계

1. 프로젝트 수행 과정
2. 아키텍처 설계
3. 기술 사양
4. ERD

3. 프로젝트 주요 기능

1. 상품 추천 로직
2. 행동로그 설계
3. 메타태그 설계

4. 시연 영상

5. 향후 개선사항

사용자 후기 중심의 뷰티 아이템 쇼핑몰 구현

타이머 기능 중심의 티켓팅 및 도서 구매 서비스 구현

▶ **접근성 중심의 상품중개 쇼핑몰 구현**

장바구니 활용을 강조한 조립식 컴퓨터 구매 쇼핑몰 구현

메신저를 연동한 선물 서비스 구현

- 접근성이라는 테마가 가지는 해석의 다양성
- 수업에서 배웠던 쇼핑몰 로직에 대한 응용

접근성 중심 = 사용자 중심

1. 그렇다면 **사용자**를 어떻게 정할 것인가?
2. 사용자 중심을 **어떤 기술**로 구현할 것인가?

01

서비스 소개

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물

활동량 및 업무량이 가장 많은

30·40대 직장인을 대상 (생활시간조사결과, 2024, 통계청 조사 결과)

30대 및 40대 직장인이 상품을 찾기 위한 시간 및 피로도를 감축할 수 있는
상품 추천 시스템을 고안, 30대 및 40대가 관심 가질만한 5개의 카테고리 [건강
/교육/여행/선물/가전] 를 토대로 **메타태그**와 **사용자 행동**을 바탕으로 하였습니다.



01

서비스 소개

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물



- Ditto · Data To you의 의미를 지님
- 상품 중개 쇼핑물로서 메타태그 × 행동로그 추천 시스템을
중점으로 둬

취향의 다차원 분해

상품을 카테고리·목적·
시즌·연령선호 등으로
태깅하여 키워드 검색보다
정교한 결과 도출

행동이 곧 신호

조회·찜·장바구니·구매·
당분간 보지 않기를
점수로 환산하여
사용자의 행동을 바로 반영

개인별 실시간 순위

같은 카테고리·목록이라도
사람마다 다른 추천 결과를
도출하여 개인화함

변화의 실시간 추적

취향이 바뀌면 추천 또한
즉각 갱신

01

팀원 소개

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물



이지수(팀장)

프로젝트 총괄 ·
회원 인증 · 마이페이지



정인혁

이커머스 기능 전반
(상품·장바구니·주문·결제)



전이레

상품 추천 시스템 엔지니어링
· 프론트엔드 기반 설계

1주

기획 · 프로젝트 규칙 정립 ·
레퍼런스 리서치

2주

API 명세 · ERD 설계 · 아키텍처 구축 · 프로토타입

3주

백엔드 핵심 기능 구현(이커머스 · 추천 시스템 엔지니어링 등)

4주

5주

6주

7주

8주

9주

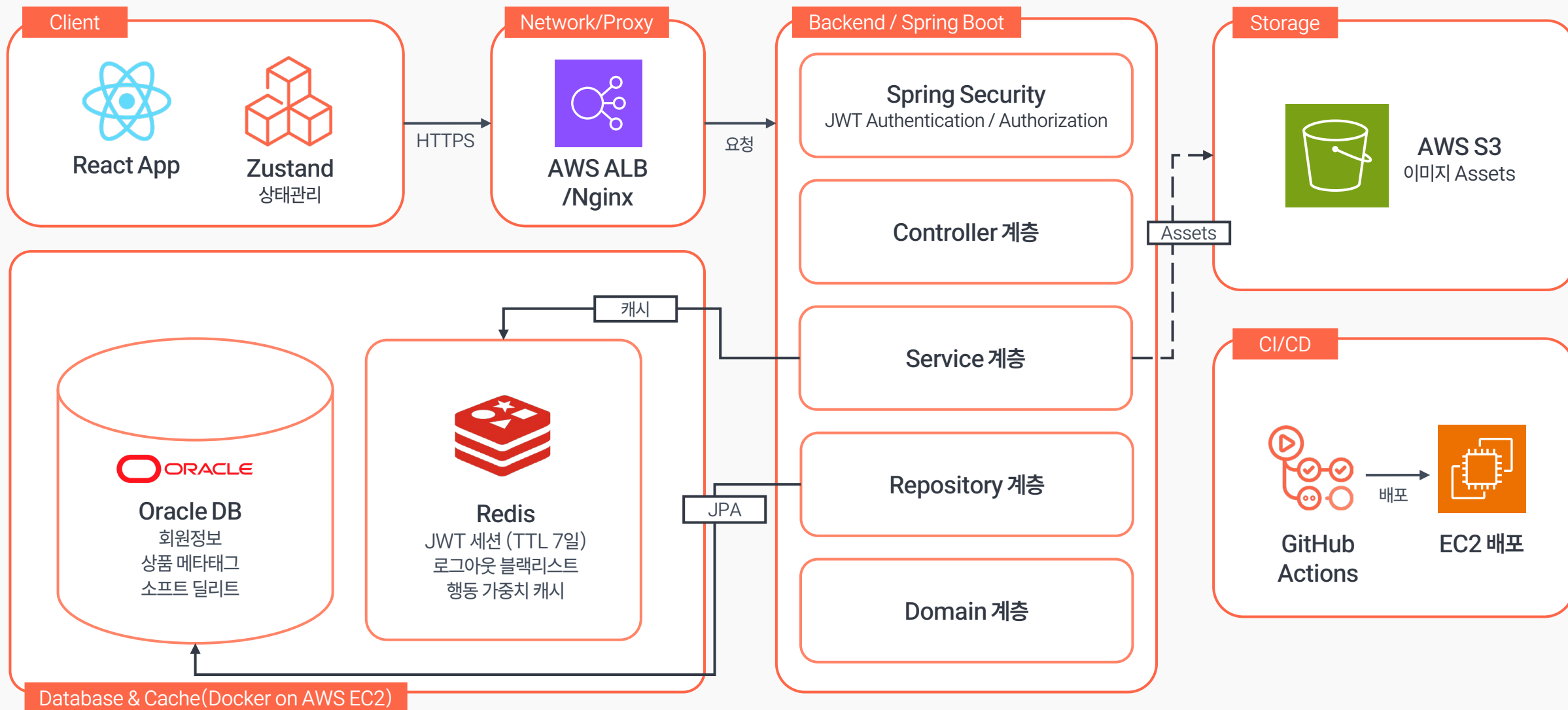
10주

백엔드 핵심 기능 구현(이커머스 · 추천 시스템 엔지니어링 등)

API 연결 및 프론트엔드 구현

통합 테스트 · 디버깅 · UI/UX 수정

AWS 통한 배포 · 발표 준비



BACK-END

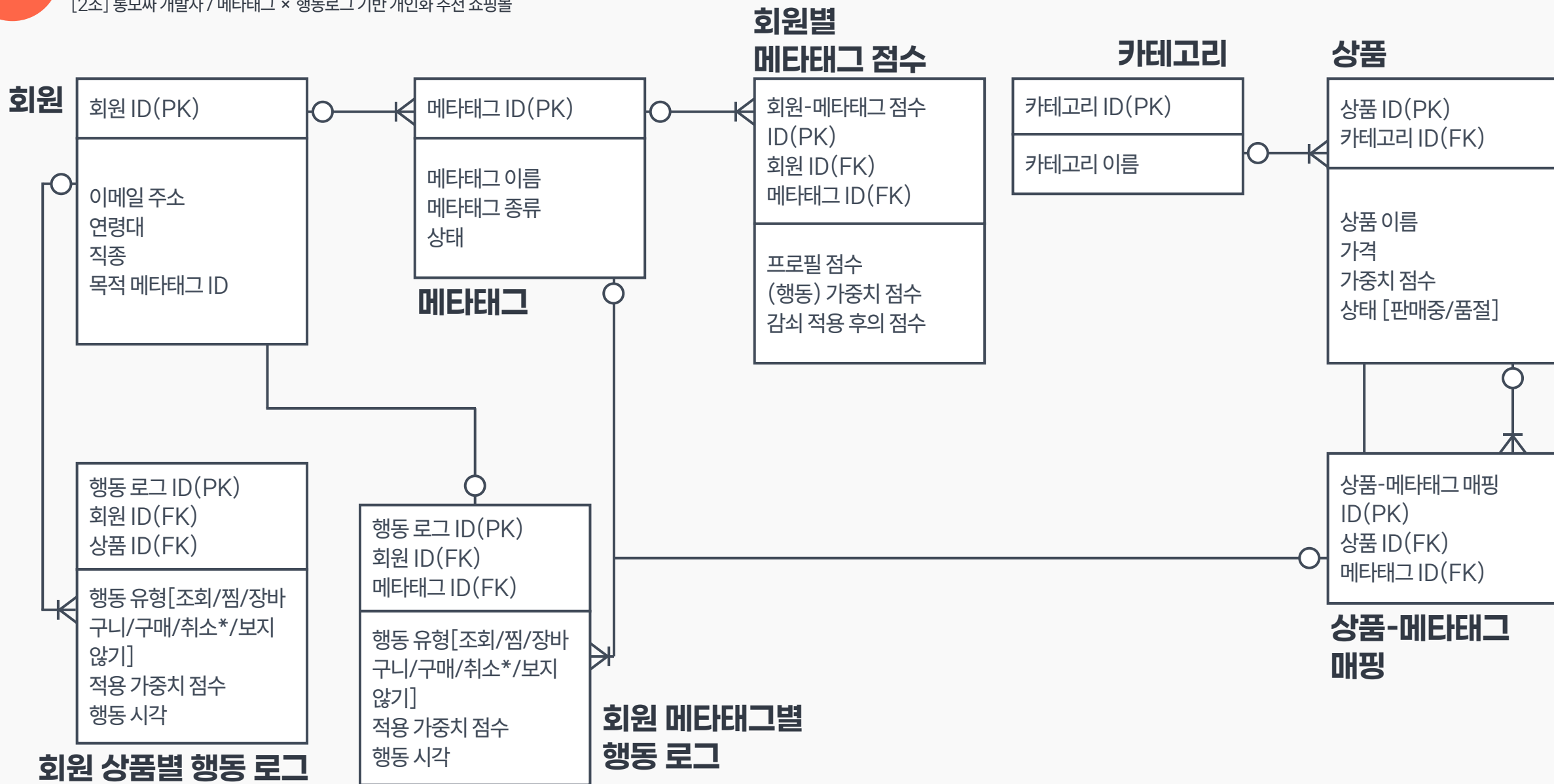
Java 17 (LTS)
Spring Boot
JPA (Hibernate)
Spring Security
JWT
Oracle
Redis

FRONT-END

JavaScript
React
React Query
Tailwind CSS

TOOLS/INFRA

GitHub
Notion
AWS
Docker



다음 3단계를 통하여 사용자는 자신이 원하는 상품을 추천 받습니다.

상품에 대한 행동

행동 가중치 점수를
메타태그에 기록

해당 메타태그를
가진 상품을
우선적으로 노출



상품 A가 가진 메타태그 A,B,C...

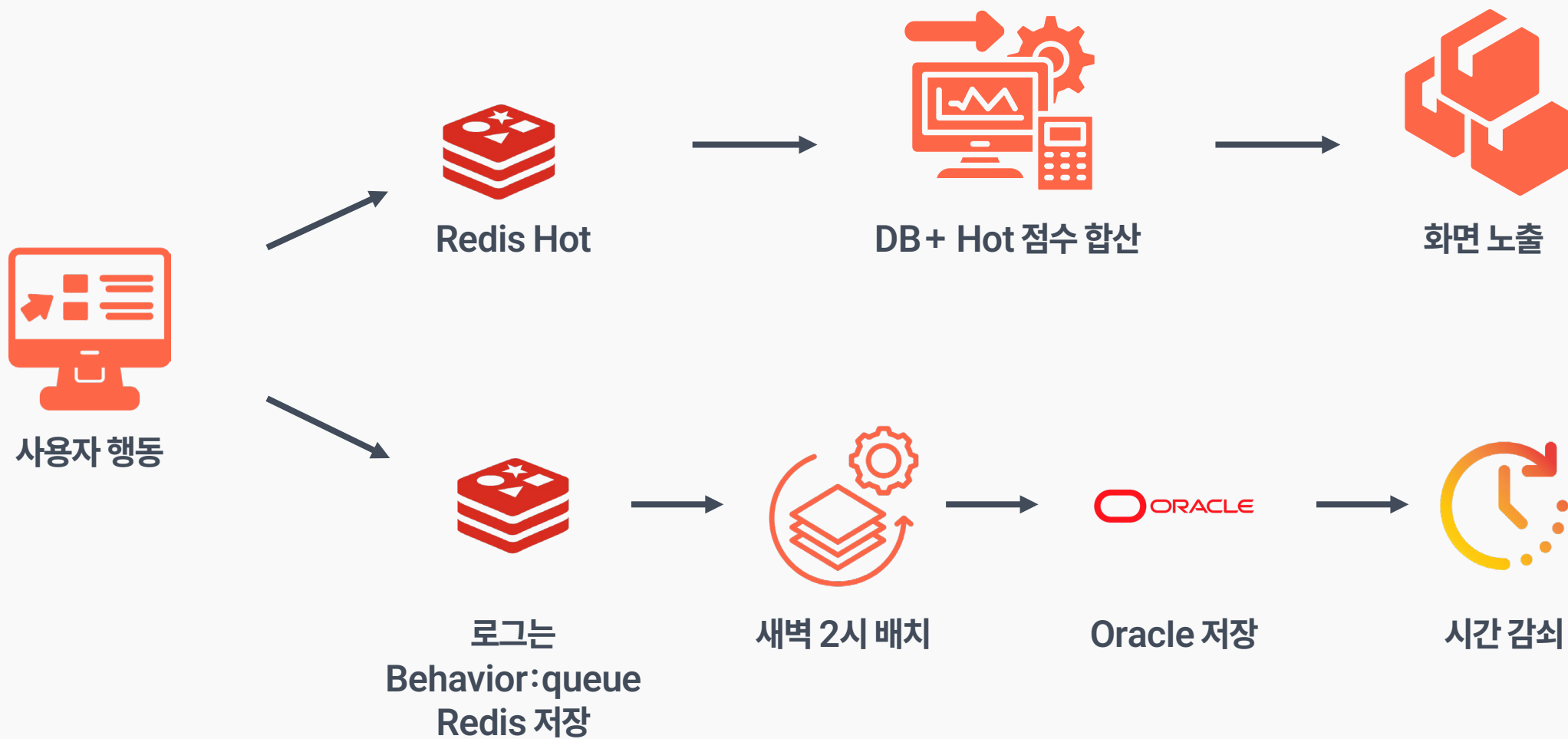


메타태그 A의 가중치 점수+
메타태그 B의 가중치 점수+
메타태그 C의 가중치 점수+
...

사용자의 각 메타태그의
점수를 합산



사용자가 선호하는 메타태그를
보유한 상품이 상단에 올라옴





조회



찜



장바구니



구매



취소



당분간
보지 않기

사용자의 6가지 행동



건강

장바구니 + 5점

선물

구매 + 10점

교육

당분간 보지 않기 -5점

각 행동에 부여된 점수에 따라
메타태그에 점수 기록



1개월 이내 원본 가중치 100%
1~2개월 사이 가중치의 70%
2~3개월 사이 가중치의 30%
3개월 초과 0점

시간에 따라 점수를 감쇠하여
사용자의 최신 관심사를 우선 반영

Table 3. User event type and weights.

Event Type	Description	Weight
View	Product viewed by clicking	1
Cart	Product added to cart	2
Purchase	Purchased product	3

... Purchases are assigned the highest weight to more accurately reflect actual preferences, while views and cart additions receive lower weights

“... 실제 유저의 선호도를 더 정확하게 반영하기 위해 **구매에 가장 높은 가중치를 부여하는 반면, 조회와 장바구니 담기에는 더 낮은 가중치를 부여한다**”

대규모 이커머스 환경에서 유사 가격 콘텐츠 기반의 하이브리드 추천 시스템
(A Hybrid Recommendation System Based on Similar-Price Content in a Large-Scale E-Commerce Environment)
– 권영오, MDPI 수록

... detail-page-view events, which are typically much denser than add-to-cart [and purchase] events ... We also correct for bias with extremely popular or on-sale items and better handle long-tail items with sparse data ...

“... **상세 페이지 조회 이벤트**는 일반적으로 **구매 이벤트**보다 훨씬 더 자주 발생한다. (이 구조에서) 추천 모델은 지나치게 인기가 많거나 할인 중인 상품에 대한 편향을 보정하고, (행동) **데이터가 희소한 상품들을 더 잘 처리하도록 설계되어...**”

Recommendations AI modeling
- Google Ads

이러한 근거에 의하여 각 행동에 따른 **가중치 점수를 책정**

행동	점수	설명
조회(VIEW)	1점	- 관심의 표현이기도 하지만 사용자의 번거로움과 경제적 위험부담은 가장 ↓ (희소성이 가장 낮음) - 단순 조회만 한 경우는 데이터 왜곡을 방지하기 위해 제외, 10초의 체류 시간과 후속 작업 처리 여부를 반영. 30분 기준 1회만 반영하여 중복·반복 조회로 인한 왜곡을 줄임
찜(BOOKMARK)	3점	- 사용자의 번거로움은 조회보다 높지만 경제적 위험부담은 ↓ (희소성 중간) - 실수로 북마크를 등록했을 가능성을 고려하여, 1분간 Redis pending에 두고 등록이 유지된 경우에 Redis에 적재
장바구니(CART)	5점	구매 직전 단계로서 사용자의 번거로움이 ↑ 하지만 경제적 위험부담은 ↓ (희소성 높음)
구매(PURCHASE)	10점	- 사용자의 번거로움과 경제적 위험부담이 가장 ↑ (희소성이 가장 높음) - 실수가 일어나지 않고, 가장 강력한 선호의 표현이며, 이후 결제 취소/환불 처리의 과정이 복잡한 것을 감안하여 바로 데이터베이스에 적용
취소(CANCEL)	취소 행동 점수 롤백	- 취소는 상품에 대한 관심을 철회/부정하는 행위(조회/당분간 보지 않기를 제외하고 적용) - 이전에 기록한 점수는 즉시 롤백 처리
당분간 보지 않기(DISLIKE)	-5점	- 추천된 상품을 유저가 직접 피드백하여 추천의 정밀도를 높임 - 당분간 보지 않기 대상이 된 상품은 한 달 동안 목록에 노출되지 않음 - 해당 상품이 가진 메타태그 점수는 감소 처리

행동 로그 설계

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물



조회
+1점

여행 +1점
가성비 +1점
나를 위한 +1점



점수 차이

조회로 인한 +3점
총 45점

조회로 인한 +2점
총 38점

행동이 발생할 경우 상품에 포함된
메타태그에 점수가 부과

메타태그 점수에 따른
노출 우선도가 달라짐

ID	MEMBER...	META_TAG_ID	PROFILE_SCORE	WEIGHT_SCORE	EFFECTIVE_SCORE	UPDATED_AT
1	1	1	20	0	0	26/06/17 14:35:38.614245000
2	1	6	20	0	0	26/06/17 14:35:38.638970000
3	1	11	20	0	0	26/06/17 14:35:38.643107000
4	1	16	5	0	0	26/06/17 14:35:38.708577000
5	1	24	5	0	0	26/06/17 14:35:38.763766000

사용자가 행동하기 전까지는 사용자의 선호를 알 수 없기 때문에(콜드 스타트), 회원 가입 시 자신의 추천 정보를 프로필로 입력하게 하여 **초기 선호도**를 분석합니다. 초기 입력에서는 사용자가 바꾸기 어려운 **직종/나이**를 제외하고는 20점의 가중치 점수가 들어갑니다.



맞춤 추천 정보

입력하시면 더 정확한 상품 추천을 받을 수 있습니다.

연령대

20대 ☒ 30대 40대 50대 60대 이상

직종

☒ 사무 영업 현장 의료 교육 기타

관심 카테고리 중복 선택 가능

건강 ☒ 교육 여행 선물 가전 모르겠어요

주로 사용하는 목적

나를 위한 구매 선물용 ☒ 가족/아이 업무/직장
취미/여가 모르겠어요

선호하는 상품 유형 중복 선택 가능

가성비 ☒ 고품질 ☒ 실용적 트렌디 친환경
모르겠어요

☒ 직종에 맞는 맞춤 상품 추천을 받겠습니다.

쇼핑을 시작하기

나중에 하기



사이트 관리자가 메타태그를 등록



판매자가 관리자가 등록한 메타태그 중 상품에 알맞은 것을 태깅



상품은 각각의 메타태그를 가지고,
해당 메타태그는 사용자마다 다른
점수를 가지게 됨

현재 8개의 **메타태그 타입**이 존재

각 메타태그 타입은 판매자가 상품에 태깅 가능한 6가지[카테고리/목적/기획의도/직종/나이대/시기별]와
시스템이 자동 등록하는 2가지[인기도/등록시기]로 나뉘어져 있음

카테고리

목적

기획의도

직종

나이대

인기도

시기별

등록시기

메타태그 설계

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물

다음은 기본적으로 등록된 메타태그로, 이 외에도 사이트 관리자는 필요하다면 **메타태그를 직접 추가**할 수 있음
한 상품은 같은 타입 내에서 혹은 다른 타입들과 병행하여 **여러 개**의 메타태그를 가질 수 있음

메타태그 타입	메타태그
카테고리(CATEGORY)	건강/교육/여행/선물/가전
목적(PURPOSE)	나를 위한 구매/선물용/가족-아이/업무-직장/취미-여가
기획의도(MERCHANDISING)	가성비/고품질/실용적/트렌디/친환경
직종(OCCUPATION)	사무/영업/현장/의료/교육/기타
나이대(AGE _ PREFERENCE)	영유아/10대/20대/30대/40대/50대/60대 이상
인기도(POPULARITY)	인기 상품/스테디셀러/주목 상품/특가 세일
시기별(SEASONAL)	초봄/봄/여름/초가을/가을/초겨울/겨울/장마/연말/연초/명절
등록 시기(RELEASE _ OR _ UPDATE)	신상품/업데이트

모든 메타태그 타입이 사용자의 선호를 동일하게 대변하기 어렵기 때문에, 메타태그가 자주 바뀌는가(변동 빈도), 유저의 경향을 반영하기 쉬운가(선호 직관성)로 기준을 두어 각 메타태그마다 가중치 점수를 다르게 부여하여 조회에서 계수로서 적용

메타태그 타입	점수	산정 이유
카테고리(CATEGORY)	1점	- 변동 빈도가 없으며 선호 직관성이 가장 높음 - 사용자가 초기 설정 가능하단 점에서 사용자의 니즈를 확실히 반영
기획의도(MERCHANDISING)	0.7점	- 변동 빈도가 높으나 직관적 선호도도 높음 - 사용자가 초기 설정이 가능하단 점에서 사용자의 니즈를 반영
목적(PURPOSE)	0.5점	변동 빈도가 낮으며 직관적 선호도가 보통
직종(OCCUPATION)	0.4점	- 변동 빈도가 낮으며 직관적 선호도가 낮음 - 다만 나이보다는 선호도 반영 가능성이 있음
나이대(AGE_PREFERENCE)	0.3점	- 변동 빈도가 낮으며 직관적 선호도 또한 낮음 - 사용자가 직접 선택할 수 없으나, 나이대에 따른 사용자 니즈가 있을 수 있음
인기도(POPULARITY)	0.2점	- 변동 빈도가 매우 높으며 직관적 선호도가 매우 낮음 - 다만 인기 있는 상품이라는 지점에서 사용자에게 추천 효용성이 있음
등록 시기(RELEASE_OR_UPDATE)	0.1점	변동 빈도가 매우 높으며 사용자 선호의 반영이 불가능함
시기별(SEASONAL)	시기에 따라 0점/ 1점	- 시기 별로 추천이 가능하단 점에서 해당 시기가 되면 1점으로 취급 - 해당 시기 외에는 0점으로 취급

개선사항 1

5대 카테고리[건강/교육/여행/선물/가전]의 취지는 좋으나
협소한 카테고리 타입으로 인하여
상품을 찾기 다소 어려움

하위 카테고리를 창설하여 좀 더 빠르게
상품 종류를 찾을 수 있도록 처리

개선사항 2

메타태그 타입의 종류가 8가지로
한정되어, 다양한 메타태그를
추가하기가 어려움

메타태그 타입을 [가격대/라이프스타일] 등으로
좀 더 다원화시켜 메타태그 정밀도를 높임

개선사항 3

임의적인 점수 체계(조회 1점, 구매 10점) 등으로 인하여
추천 적합성이 보증되지 않음

취소 로그·환불 사유 등 데이터 축적 후, 데이터 분석 시스템과
연계해 점수 자동 조정

개선사항 4

판매자가 메타태그를 올바르게 올리지 않게
상품에 태깅할 경우에 대한 대책이 미비

1. 상품의 경우 바로 등록이 아니라 관리자를 통한 승인 과정을 거쳐, 메타태그가 이상하거나 할 경우 반려 가능
2. 권장 태그 세트 등을 통하여 카테고리-메타태그 등록 과정을 간편화하여 판매자의 등록 부담과 위험 수준을 낮추고 관리자 차원에서 정밀성을 높임
3. 관리자가 메타태그-상품 매핑에 관한 모니터링이 가능한 대시보드 추가, 그리고 해당 메타태그의 상품과 총 가중치 점수의 비율을 파악

Q&A

효율적인 로그 기록 - 배치Batch 사용

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물

사용자의 수가 늘어나면 각 행동에 대한 로그가 많이 쌓이므로 DB에 부담
이에 따라 **배치**를 사용하여 일괄 등록



Redis에 사용자 로그들을 축적



사용자 이용이 적은 오전 2시에
1000건씩 일괄 처리



DB에 등록

북마크/장바구니/구매의 경우



재행동에 따라 점수가 가속화될 경우 오히려 특정 카테고리/상품으로의 추천이 강해질 우려가 있어 설계에서 수정하지 않기로 함
이후 로그 분석 등을 통하여 도입을 고려하고 있음

조회시 경우



단순 클릭 어뷰징으로 인한 점수 왜곡을 방지해야 하지만, 완전한 차단은 오히려 사용자의 관심을 제대로 반영하지 않을 수도 있음. 이에 따라 행동 횟수별로 차등 가산하는 형태로 재행동을 처리

- 1~5회 회당 1.0점(기본 점수)
- 6~10회 회당 0.5점(기본 점수의 50%만 반영)
- 11회~ 회당 0.1점 가산

```
//재구매 시기가 되면 updatedAt을 끌어올린다
@Transactional
public void applyUpliftFromUpdatedAt() {
    LocalDateTime now = LocalDateTime.now();
    List<MemberTagWeight> weights = memberTagWeightRepository.findAll();
    for (MemberTagWeight weight:weights){
        Long memberId = weight.getMember().getId();
        Long metaTagId = weight.getMetaTag().getId();
        LocalDateTime updatedAt = weight.getUpdatedAt();

        //30일 미만이라면 생각한다
        if (updatedAt == null || ChronoUnit.DAYS.between(updatedAt, now) < 30) continue;

        Duration interval = resolveAveragePurchaseInterval(memberId, metaTagId);
        LocalDateTime windowStart = updatedAt.plus(interval).minusDays(7);

        if (!now.isBefore(windowStart)) { // now >= windowStart
            memberTagWeightRepository.touchUpdatedAt(weight.getId(), now); // updatedAt 갱신
        }
    }
}
```

상품 구매의 주기를 무시하고 **무작정 감소**하는 것은 실제 사용자의 선호와 동떨어짐
따라서 사용자의 구매 시기 간격에 맞춰 시간 감소로 인하여 감소한 **점수를 복구**

시즌성 상품의 경우도 마찬가지로 해당 시즌이 되면 점수를 복구 처리

가중치 조회 시 로직

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물

실제 상품 조회시에는 다음과 같은 수식으로 계산이 됨

$$\text{상품점수} = \sum_{t \in \text{상품태그}} [(\text{프로필}_t + \text{행동}_t^{\text{감쇠}} + \text{hot}_t) \times \text{계수}_{\text{type}(t)}]$$

가중치 조회 시 로직

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물

```
// 상품마다 보유 태그 점수를 합산
double score = 0;
for (Long tagId : tagIds) {
    score += tagScoreMap.getOrDefault(tagId, 0.0);
}

// 점수 내림차순, 동점은 누적 가중치로 2차 정렬
products.sort(Comparator
    .comparingDouble(p -> productScoreMap.get(p.getId()))
    .reversed()
    .thenComparing(Product::getWeightScore, nullsLast(reverseOrder())));
```

실시간 추천의 병목을 막기 위해 **2단계로 분리**

- 회원의 태그 가중치를 **Map<태그ID, 점수>**로 메모리에 적재 (최악의 경우라도 태그는 십 수 개로 한정 됨, 1회 조회)
- 상품이 가진 태그(보통 3~5개)를 맵에서 조회해 더함 → $\text{HashMap.get()} = O(1)$

대량의 데이터베이스 JOIN 처리 없이, 상품 N개 × 태그 k개의 단순 메모리 합산으로 정렬 완료

당분간 보지 않기에 대한 메타태그 감소 계수치

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물

당분간 보지 않기 처리에 따른 메타태그 감소 계수치는 다음과 같습니다.
많은 상품을 보유하는 메타태그일 수록 낮게, 사용자의 선호도를 크게 반영할수록 높게 산정하였습니다.

메타태그 타입	점수	산정 이유
목적(PURPOSE)	0.9점	사용자의 의향이 강하게 반영됨 태그 변경 시 추천 결과에 즉각적이고 큰 영향을 미침
기획의도(MERCHANDISING)	0.4점	판매자의 기획적 의도가 반영되며 사용자의 성향과 합치함
카테고리(CATEGORY)	0.3점	사용자의 의향이 강하게 반영되나, 가장 넓은 분류 범위를 가지고 있음 상품 보유량이 많아 과도한 감소를 방지하기 위해 가중치 조절
직종(OCCUPATION)	0.1점	직접적인 상품 선호도와의 연관성이 낮음
나이대(AGE _ PREFERENCE)	0.1점	직접적인 상품 선호도와의 연관성이 낮음
인기도(POPULARITY)	0점	사용자의 상품 선호도와의 연관성이 없음
등록 시기 (RELEASE _ OR _ UPDATE)	0점	사용자의 상품 선호도와의 연관성이 없음
시기별(SEASONAL)	0점	시기에 따라서 다르게 나타나는 특수한 태그이므로 처리하지 않음

메타태그 어뷰징 방지

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물

메타태그를 합산하여 추천하는 설계 특성상 판매자가 자신의 판매 품목을 상위에 두기 위해 메타태그를 전부 넣어두는 **어뷰징**이 발생할 수 있음

이에 따라 다음과 같이 카테고리 타입마다 등록 상한선을 두어 어뷰징을 방지

메타태그 타입	등록 상한선
카테고리(CATEGORY)	1개
직종(OCCUPATION)	2개
나이대(AGE_PREFERENCE)	2개
기획의도(MERCHANDISING)	5개
목적(PURPOSE)	3개
시기별(SEASONAL)	2개

※ 인기도, 등록시기의 경우 시스템에서 자동으로 등록하는 메타태그이므로 판매자가 등록이 불가능
이에 따라 상한선이 별도로 존재하지 않음

Redis key

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물

본 프로젝트 주요 기능인 행동로그 및 점수 체계에 사용된 Redis key는 다음과 같습니다

키명	Key 형식	자료 구조	용도 및 해설
weight:hot	weight:hot:{memberId}	Hash	- 회원별 실시간 행동 점수 버퍼 - field={metaTagId}, value=누적 delta(문자열 숫자). 조회·찜·장바구니·dislike 시 즉시 HINCRBY 처리. 추천 API는 DB 점수 + hot delta를 합산. 새벽 배치에서 member_tag_weights에 merge 후 키 삭제.
behavior:queue	behavior:queue	List	- 행동 로그 DB 저장 대기 큐 - value=ProductWeightLog JSON. VIEW·DISLIKE 등은 LPUSH, 배치는 RPOP. 새벽 2시에 flush.
cart:pending	cart:pending	Hash	- 장바구니 담기 DB 확정 전 임시 저장 - field={memberId}:{productId}, value=로그 JSON. 장바구니 취소 시 field 삭제. 배치에서 DB 저장 후 삭제.
bookmark:pending	bookmark:pending	Sorted Set (ZSET)	- 찜 1분 지연 확정용 - member={memberId}:{productId}, score=찜 시각(ms). 1분 경과 후 점수 반영 + behavior:queue 적재. 찜 취소 시 member 제거.
dislike:hide	dislike:hide:{memberId}	Sorted Set (ZSET)	- 당분간 보이지 않기 숨김 목록 - member={productId}, score=만료 시각(ms, 현재+30일). 목록 조회 시 score ≥ now인 상품 필터링. 장바구니·구매 시 해당 productId 제거.
view:dedup	view:dedup:{memberId}:{productId}	String	- 조회 중복 방지 - value="1", TTL 30분. 30분 내 동일 상품 재조회는 점수 미반영.

AI Collaborative Filtering과의 비교

[2조] 통모짜 개발자 / 메타태그 × 행동로그 기반 개인화 추천 쇼핑물



AI 협업 필터링 (CF)

- 초기 장벽 데이터 희소성에 취약하여 서비스 초기 추천 불가능
- 추천 사유 설명 불가능 특정 상품이 왜 추천되었는지 인과관계 추적 불가
- 높은 비용 ML 파이프라인(GPU 서버, 실시간 서빙 등) 구축으로 인한 인프라 비용



D-TO 메타태그 기반 서비스

- 초기 장벽 해소 가입 시 초기 프로필 데이터 분석을 통해 콜드 스타트를 해소
- 추천 사유 설명 가능 실시간 가중치 합산을 통하여 인과관계 추적 가능
- 비용 절감 WAS 메모리 단의 HashMap 매핑으로 추가 서버 비용 없이 실시간 연산

감사합니다

2조 통모짜 개발자

이지수 전이레 정인혁